Plan de construcción — Control de viáticos

Equipo: Mango, Fabián y Dubán. **Base:** enunciado, rúbrica y explicación grabada del profesor (14/10/2026).

1. Objetivo y condiciones de entrega

La aplicación evaluada debe abrir en Visual Studio Community y construirse en C#, .NET Framework 4.7.2 o superior y WinForms. Puede tener una navegación visual parecida a una página web, pero es una aplicación de escritorio. El prototipo HTML sirve como referencia de apariencia y recorrido. No se deben agregar paquetes NuGet; el equipo debe poder explicar el código y las técnicas que utilice. [V 02:56–05:18; 06:36–07:30; 09:42–10:31]

Los registros se manejan con objetos y listas durante la ejecución. La aplicación debe permitir guardar, consultar y modificar esos datos en memoria; el profesor no solicita una conexión con base de datos. [V 14:51–15:31; 42:18–42:47]

2. Qué software hay que construir

Una aplicación de escritorio para Los 3 Patitos S.A. donde se entra con usuario y clave; se administran usuarios, empleados y tipos de viático; se registra una liquidación de un viaje con varios conceptos; y se consultan reportes y la bitácora. Los datos se manejan mediante **objetos y listas**. Cada pantalla de registro debe permitir **crear, consultar, modificar y borrar**. Cada objeto debe tener su listado visual correspondiente. [V 27:47–29:29; 30:54–37:21; 42:18–44:43; 51:50–52:38]

Recorrido exacto que debe funcionar

1. **Entrada y menú:** usuario activo entra. El usuario normal usa empleados, tipos de viático, liquidaciones y reportes. Solo el administrador gestiona usuarios y ejecuta la carga inicial; el administrador también puede usar todas las demás funciones. Usuario inactivo no entra, aunque acierte la clave. [V 30:54–33:27; 47:09–48:21]
2. **Usuarios:** identificación, nombre completo, nombre de usuario, clave, tipo (normal/administrador) y estado. Se muestran en una tabla visual. Hay que poder crear, consultar, modificar y borrar. [V 33:38–35:00; 51:50–52:14]
3. **Empleados:** identificación, cédula, nombre, dos apellidos, teléfono, dirección y estado. Un empleado es distinto de un usuario del sistema. Se muestran y se operan igual que usuarios. [V 35:10–36:02; 47:09–48:04]
4. **Tipos de viático:** identificador, nombre o categoría, monto y estado. Un tipo es el concepto (desayuno, almuerzo, cena, hospedaje, etc.); **no es** la liquidación. Se muestran y se operan igual. [V 36:02–37:21]
5. **Liquidación:** cabecera con consecutivo, empleado, inicio y fin con fecha y hora, y justificación; detalle con uno o más tipos de viático y sus montos asignados. El total se obtiene sumando detalles. Crear, consultar, modificar y borrar deben mantener cabecera y detalles coherentes. Si el empleado o el viático están inactivos, no se pueden elegir para una liquidación nueva. [V 37:35–42:13; 47:09–48:21]
6. **Bitácora:** lista separada para dejar quién hizo cada operación, cuándo, en qué sección y qué movimiento hizo. No meter el historial de cambios dentro del objeto empleado o viático. [V 42:58–44:43]
7. **Reportes:** (a) consumos por rango de fechas: empleado, fecha, viático, monto y total; (b) consumos de un empleado: fecha, viático, monto y total; (c) operaciones filtradas por usuario y fechas: usuario, fecha, sección y movimiento. Los filtros deben funcionar, no solo aparecer dibujados. [V 44:51–46:59; 54:03–55:18]

Carga inicial y horarios

La carga inicial es una **opción** que ejecuta el administrador, nunca algo que ocurre automáticamente al abrir. Debe insertar datos en tres listas: usuarios, empleados y tipos de viático. Debe dejar agregar otros registros después y no estropear los existentes. Los tipos iniciales son desayuno ¢3 000, almuerzo ¢5 000 y cena ¢5 000. [V 13:18–16:20; 48:33–48:53; 52:47–53:13]

Las reglas que aplicará el equipo son: desayuno, salida **≤ 07:00** y llegada **≥ 10:00**; almuerzo, salida **≤ 10:00** y llegada **≥ 14:00**; cena, salida **≥ 16:00** y llegada **≥ 20:00**, siguiendo la explicación oral del profesor. Las comparaciones incluyen la hora exacta del límite. Para un viaje de varios días, una sola liquidación puede incluir varios conceptos; el equipo debe acordar con ejemplos cómo evaluar cada día intermedio antes de programar esa parte. [V 41:22–42:13; 48:59–49:47]

3. Acuerdos de funcionamiento antes del código

Fabián publica un documento corto llamado **Contrato del proyecto** y los tres lo aprueban en GitHub. Contendrá:

- Nombres de los objetos y campos; qué datos corresponden a cabecera y cuáles a detalle; qué listas existen.
- Cómo se identifica cada registro y qué pasa con un registro relacionado cuando se intenta borrar. Se debe conservar coherencia para reportes y bitácora.

- Cómo se crea el **primer administrador** sin ejecutar automáticamente toda la carga inicial. Es una decisión necesaria para poder entrar al sistema. Una opción es una configuración inicial explícita que solo crea esa cuenta y deja la carga de ejemplo para después; se documenta y se prueba antes de avanzar.
- Qué guarda la bitácora y cuándo se añade una entrada (solo después de una operación completada).
- Qué fecha usa el reporte de consumo. Si se decide guardar fecha de consumo por detalle para viajes de varios días, incluirla en el contrato y usarla en los dos reportes.
- Qué pasa con montos históricos si se edita el precio del catálogo. Para que el reporte mantenga el monto realmente asignado, el detalle debe guardar una copia del monto aplicado al registrarlo.
- Ejemplos escritos de los límites de desayuno, almuerzo y cena, y de un viaje de dos días con varios detalles.

Caso que el equipo debe definir con un ejemplo: cómo aplicar las ventanas de comida a cada día de un viaje de varios días. Si el ejemplo no permite resolverlo con lo explicado en clase, anoten una pregunta concreta para el profesor. [V 41:22–42:13]

4. Quién hace qué, sin pisarse

Integrante	Responsabilidad principal	Entrega que libera a otro	Revisión cruzada
Fabián	Estructura del proyecto en Visual Studio, objetos/listas compartidos, inicio de sesión, permisos, usuarios, carga inicial, bitácora e integración.	Contrato aprobado y proyecto que abre en las tres computadoras; operaciones comunes para los demás.	Revisa integración de Mango y Dubán.
Mango	Diseño visual y navegación WinForms; pantallas completas de empleados y tipos de viático; visualización y filtros de los tres reportes.	Guía de controles, catálogos listos y pantallas de reporte listas para recibir resultados.	Revisa claridad de las pantallas de Fabián y Dubán.
Dubán	Cabecera y detalle de liquidación, validaciones de estado/fechas/horas, total y consultas de consumo de los dos primeros reportes.	Operaciones de liquidación y resultados que Mango puede mostrar.	Revisa reglas de datos compartidas y casos límite.

Fabián coordina las integraciones, **pero no programa el trabajo de todos**. Cada responsable prepara sus cambios y otro integrante los revisa. Para una pantalla ajena, Mango propone ajustes visuales en una tarea; el dueño aplica el cambio o acuerdan una entrega puntual.

5. Orden de construcción y entregas verificables

Paso 0 — Arranque del equipo

Fabián crea el repositorio y una solución WinForms mínima que abra en Visual Studio Community con el framework exigido. Configura para ignorar archivos de compilación y archivos personales del IDE. Abre el tablero Pendiente → En curso → Revisión → Terminado. **Mango y Dubán** descargan y abren exactamente esa solución. **Listo cuando:** los tres muestran la ventana inicial en sus equipos.

Paso 1 — Contrato y ejemplo común

Los tres completan los acuerdos del apartado 3. **Mango** dibuja login, menú, cuatro registros y tres reportes con los campos exactos. **Dubán** define un ejemplo de viaje con cena, hospedaje y desayuno en la misma liquidación, más casos en las horas límite. **Fabián** cierra nombres de objetos/listas y flujo de permisos. **Listo cuando:** el ejemplo puede explicarse de principio a fin sin adivinar datos ni reglas.

Paso 2 — Objetos, listas y primer acceso

Fabián incorpora los objetos y listas comunes y un primer administrador explícito. Construye login y menú según rol, y una operación mínima de guardar/consultar. **Mango** entrega navegación visual y empieza empleados; **Dubán** prepara cabecera/detalle y las validaciones con datos de ejemplo. **Integrar primero la base de Fabián. Listo cuando:** se puede entrar como administrador, un usuario inactivo no entra, y un registro añadido a una lista aparece en su tabla visual durante la ejecución.

Paso 3 — Registros completos y bitácora

Mango termina CRUD de empleados y tipos de viático en dos entregas separadas. **Fabián** termina CRUD de usuarios, permisos y bitácora central. **Dubán** termina operaciones de cabecera/detalle usando los objetos aprobados. Los tres conectan sus operaciones a la bitácora sin editar el mismo archivo al mismo tiempo. **Integrar usuarios, empleados y viáticos antes de conectar la liquidación final. Listo cuando:** cada registro se crea, consulta, modifica y borra; los estados se respetan; cada operación completa queda en bitácora.

Paso 4 — Liquidación completa

Dubán une pantalla y lógica: empleado activo, inicio/fin, justificación, varios detalles y total. Aplica las reglas de horarios a desayuno, almuerzo y cena; para cena usa salida a las 16:00 o después y llegada a las 20:00 o después. Prueba horas exactas y un minuto fuera. **Mango** revisa que agregar, editar y quitar detalles sea claro. **Fabián** comprueba permisos y que la operación de cabecera/detalle no deje datos a medias. **Listo cuando:** se crea una liquidación de varios conceptos, se consulta, modifica y borra conservando total y vínculos; las tres reglas de horarios funcionan.

Paso 5 — Carga inicial y tres reportes

Fabián termina la opción manual de carga para usuarios, empleados y tipos, protegida para administrador. **Dubán** entrega los datos calculados de consumos por fechas y por empleado. **Fabián** entrega la consulta de bitácora por usuario y fechas. **Mango** conecta los tres filtros y tablas. **Listo cuando:** antes de accionar la carga no aparecen datos de ejemplo; después se ven los tres tipos iniciales; se pueden seguir agregando registros; cada reporte filtra, muestra las columnas pedidas y totaliza donde corresponde.

Paso 6 — Rúbrica, informe y exposición

Los tres recorren todos los rubros de la rúbrica en la aplicación abierta desde una copia limpia del repositorio. **Mango** corrige navegación y presentación; **Dubán**, cálculos y restricciones; **Fabián**, acceso, bitácora e integración. Cada quien anota un problema real y cómo lo resolvió. **Fabián** arma el informe único y los otros dos revisan formato y contenido. El informe debe ocupar **una página completa como mínimo y dos como máximo**, con márgenes estándar, letra Calibri, Times New Roman o Tahoma de 12 puntos y dos columnas. Debe cubrir diseño, problemas con sus soluciones y lecciones aprendidas. Ensayan una defensa donde **cada integrante** entra al sistema, ejecuta una función y explica el código que le tocó y al menos una función ajena. **Listo cuando:** proyecto, informe y demostración se pueden entregar sin arreglos de último minuto. [V 16:30–21:39; 50:13–51:41]

6. GitHub: cómo evitar choques

1. Una tarea = un responsable, archivos propios, dependencia visible y una prueba de aceptación. Ejemplos: usuarios-crud, empleados-crud, liquidacion-detalles, reporte-usuario.
2. Una rama corta por tarea. No se sube trabajo incompleto a la rama principal. Al cambiar un objeto o una lista compartida, se integra ese contrato antes de que otras ramas dependan de él.
3. Antes de integrar, otro integrante comprueba: abre en Visual Studio, ejecuta el caso descrito, verifica que no se añadieron NuGet y deja un comentario breve de revisión.
4. El dueño del archivo resuelve el primer intento de conflicto; Fabián ayuda si afecta varias secciones. Después de cada integración compartida, los otros actualizan sus ramas.
5. Actualización diaria de una línea por tarea: hecho / siguiente / bloqueo. Una pregunta concreta se deja en la tarea. Solo se reúnen cuando un ejemplo escrito no resuelve el bloqueo.

7. Pruebas que deben mostrar al profesor

Demostración	Resultado esperado
Login de administrador, normal e inactivo	Dos menús según rol; inactivo rechazado.
CRUD de usuarios, empleados y viáticos	Crear, consultar, modificar y borrar en cada lista y tabla visual.
Empleado o viático inactivo	No se acepta para liquidación nueva.
Una salida con cena, hospedaje y desayuno	Una cabecera, tres detalles, monto correcto y consulta posterior.
Horas exactamente en el límite y un minuto fuera	Se aplica la regla acordada y el resultado se explica.
Carga inicial manual	No ocurre al abrir; solo administrador; inserta en tres listas y permite nuevos registros.
Reportes con al menos dos empleados y dos usuarios	Cada filtro modifica resultados; se ven las columnas y totales pedidos.
Cambio hecho por dos usuarios distintos	Bitácora separa autor, fecha/hora, sección y movimiento.
Copia limpia del repositorio	Abre y ejecuta en Visual Studio sin instalar NuGet.

8. Plazos y entrega

El profesor explica que **Grupo 1 entrega el 2/11/2026 a las 08:00** y **Grupo 2 el 4/11/2026 a las 08:00**; el plazo corresponde al grupo, no a un rango libre del 2 al 4. La exposición es el día asignado, en una cita entre 08:00 y mediodía. Confirmen a qué grupo pertenecen y la cita exacta; suban la entrega antes de las 08:00 del día que corresponda. [V 01:30–02:32; 07:30–09:35]

Hito interno	Resultado
16 oct.	Repositorio que abre en tres equipos y contrato aprobado.
21 oct.	Objetos/listas, login y primeros CRUD integrados.
25 oct.	Cuatro registros completos; liquidación con varios detalles.
28 oct.	Carga inicial, bitácora y reportes integrados.
30 oct.	Rúbrica demostrada; informe en formato; defensa ensayada.
31 oct.	Versión final lista para entregar con margen antes del primer plazo.

Prioridad si el tiempo aprieta: abrir sin dependencias extra; cubrir los cuatro CRUD, en especial las operaciones dobles de liquidación; permisos y estados; carga inicial manual; horarios; los tres reportes; objetos/listas; comentarios útiles en código; informe y presencia de los tres. La apariencia debe ser clara y consistente, pero no sustituye ninguna función de la rúbrica.

Fuentes y notas de interpretación

- **V** = Transcripcion_video_explicacion_proyecto.txt, con marcas de tiempo. Es automática; una frase dudosa se comprueba en el MP4 original.
- ITI 321 - Proyecto I.docx: enunciado y restricciones escritas.
- Rubrica I Proyecto.xlsx: lista de rubros y puntos.